

Programs Covered

1. Quick Sort
2. Merge Sort
3. Heap Sort
4. Fractional Knapsack (Greedy Method)
5. 0/1 Knapsack using Dynamic Programming
6. Dijkstra's Algorithm (Single Source Shortest Path)
7. Prim's Algorithm (Minimum Cost Spanning Tree)
8. N-Queens Problem (Backtracking)
9. Floyd-Warshall Algorithm (All Pairs Shortest Path)
10. Strassen's Matrix Multiplication
11. Kruskal's Algorithm (Minimum Cost Spanning Tree)
12. Ford-Fulkerson Algorithm (Maximum Flow)
13. Sum of Subsets Problem (Backtracking)
14. Hamiltonian Cycle (Backtracking)
15. Job Sequencing with Deadlines (Greedy Method)
16. Topological Sorting (using DFS / Stack)
17. Graph Traversal using BFS and DFS
18. Topological Sorting using BFS (Kahn's Algorithm)
19. Travelling Salesman Problem (Dynamic Programming)

Programs Covered

Quick Sort

ALGORITHM: QUICK SORT

QUICKSORT(A, low, high)

1. If low >= high
Return

2. Set key = A[low] // Pivot element

3. Set i = low + 1
Set j = high

4. While i <= j do

While i <= high AND A[i] <= key
i = i + 1

While A[j] > key
j = j - 1

```

If i < j
Swap(A[i], A[j])
5. End While

6. Swap(A[low], A[j]) // Place pivot at correct position

7. QUICKSORT(A, low, j - 1)

8. QUICKSORT(A, j + 1, high)

9. Return

```

MAIN PROGRAM

```

1. Read n

2. Read array A[0...n-1]

3. Call QUICKSORT(A, 0, n-1)

4. Print the sorted array

```

2. Merge Sort

ALGORITHM: MERGE SORT

MERGE(A, low, mid, high)

```

1. Set i = low
Set j = mid + 1
Set k = 0

2. While i <= mid AND j <= high

If A[i] <= A[j]
temp[k] = A[i]
i = i + 1
Else
temp[k] = A[j]
j = j + 1

k = k + 1

3. While i <= mid

temp[k] = A[i]
i = i + 1
k = k + 1

4. While j <= high

temp[k] = A[j]
j = j + 1
k = k + 1

5. Copy temp[0...k-1] into A[low...high]

```

MERGESORT(A, low, high)

```

1. If low < high

```

$mid = (low + high) / 2$

2. MERGESORT(A, low, mid)

3. MERGESORT(A, mid + 1, high)

4. MERGE(A, low, mid, high)

5. Return

3. Heap Sort

ALGORITHM: HEAP SORT

HEAPIFY(A, n, i)

1. Set largest = i

2. Set left = $2 \times i + 1$

3. Set right = $2 \times i + 2$

4. If left < n AND A[left] > A[largest]
largest = left

5. If right < n AND A[right] > A[largest]
largest = right

6. If largest $\neq$ i

Swap(A[i], A[largest])

HEAPIFY(A, n, largest)

7. Return

HEAPSORT(A, n)

1. For i = $(n/2) - 1$ down to 0

HEAPIFY(A, n, i)

2. For i = n - 1 down to 1

Swap(A[0], A[i])

HEAPIFY(A, i, 0)

3. Return

4. Fractional Knapsack (Greedy Method)

ALGORITHM: FRACTIONAL KNAPSACK

FRACTIONAL_KNAPSACK(weights[], values[], n, capacity)

1. For i = 0 to n - 1

ratio[i] = values[i] / weights[i]

2. Sort all items in descending order of ratio

3. Set totalValue = 0

4. For each item in sorted order

```

If weights[i] <= capacity

totalValue = totalValue + values[i]

capacity = capacity - weights[i]

Else

totalValue = totalValue +
values[i] × (capacity / weights[i])

capacity = 0

Break

5. Return totalValue

```

5. 0/1 Knapsack using Dynamic Programming

01_KNAPSACK(weights[], values[], n, W)

1. Create dp[0...n][0...W] and initialize all elements to 0.

2. For i = 1 to n

For j = 0 to W

If weights[i - 1] <= j

```

dp[i][j] = MAX(values[i - 1] +
dp[i - 1][j - weights[i - 1]],
dp[i - 1][j])

```

Else

```
dp[i][j] = dp[i - 1][j]
```

3. Set maxProfit = dp[n][W]

4. Set j = W

5. For i = n down to 1

If dp[i][j] ≠ dp[i - 1][j]

Mark item i as selected

```
j = j - weights[i - 1]
```

6. Return maxProfit and selected items

MAIN PROGRAM

1. Read n

2. Read weights[]

3. Read values[]

4. Read W (Knapsack Capacity)

5. Call 01_KNAPSACK(weights, values, n, W)

6. Print DP table

7. Print maximum profit

8. Print selected items

6. Dijkstra's Algorithm (Single Source Shortest Path)

ALGORITHM: DIJKSTRA'S ALGORITHM (Shortest Path)

DIJKSTRA(cost[][] , n, source)

1. For i = 1 to n

dist[i] = cost[source][i]

flag[i] = 0

2. Set flag[source] = 1.

3. Set dist[source] = 0.

4. Set count = 2.

5. While count <= n do

a. Find the vertex u having the minimum dist[u]
such that flag[u] = 0.

b. Set flag[u] = 1.

c. count = count + 1.

d. For each vertex w such that flag[w] = 0

If dist[u] + cost[u][w] < dist[w] then

dist[w] = dist[u] + cost[u][w]

6. Return the distance array (dist[]).

7. Prim's Algorithm (Minimum Cost Spanning Tree)

PRIM(cost[][] , n)

1. Set visited[1] = 1.

2. Set all other visited vertices to 0.

3. Set mincost = 0.

4. Set edgesAdded = 0.

5. While edgesAdded < n - 1 do

a. Set min = INFINITY.

b. For i = 1 to n

For j = 1 to n

If visited[i] = 1 AND visited[j] = 0 AND cost[i][j] < min then

min = cost[i][j]

a = i

b = j

c. Print the edge (a, b) and its cost (min).

d. mincost = mincost + min.

e. Set visited[b] = 1.

f. Set cost[a][b] = INFINITY.

g. Set cost[b][a] = INFINITY.

h. edgesAdded = edgesAdded + 1.

6. Print the total minimum cost (mincost).

8. N-Queens Problem (Backtracking)

ALGORITHM: N-QUEENS (Backtracking)

PLACE(k, i)

1. For $j = 1$ to $k - 1$

If $a[j] = i$ OR $|a[j] - i| = |j - k|$ then

Return FALSE

2. Return TRUE

NQUEENS(n)

1. Set $k = 1$.

2. Set $a[k] = 0$.

3. While $k \neq 0$ do

a. Set $i = a[k] + 1$.

b. While $i \leq n$ AND PLACE(k, i) = FALSE do

$i = i + 1$.

c. If $i \leq n$ then

$a[k] = i$.

If $k = n$ then

Print the current solution.

Else

$k = k + 1$.

$a[k] = 0$.

d. Else

$k = k - 1$. // Backtrack

4. Repeat until all solutions are found.

9. Floyd-Warshall Algorithm (All Pairs Shortest Path)

ALGORITHM: FLOYD-WARSHALL ALGORITHM (All-Pairs Shortest Path)

FLOYD_WARSHALL(D[][], n)

1. For $k = 0$ to $n - 1$

For $i = 0$ to $n - 1$

For $j = 0$ to $n - 1$

If $D[i][k] + D[k][j] < D[i][j]$ then

$D[i][j] = D[i][k] + D[k][j]$

2. Return the updated distance matrix $D[][]$.

MAIN PROGRAM

1. Read the number of vertices (n).
 2. Read the adjacency matrix (D[][]).
 3. For every pair of vertices (i, j)
- If there is no edge between i and j and $i \neq j$
- Set $D[i][j] = \text{INFINITY}$.
4. Call:
FLOYD_WARSHALL(D, n)
 5. Display the all-pairs shortest path matrix

10. Strassen's Matrix Multiplication

ALGORITHM: STRASSEN'S MATRIX MULTIPLICATION

STRASSEN(A[], B[], n)

1. If $n = 1$ then
- Return $A \times B$.
2. Partition matrix A into four sub-matrices:
 $A_{11}, A_{12}, A_{21}, A_{22}$.
 3. Partition matrix B into four sub-matrices:
 $B_{11}, B_{12}, B_{21}, B_{22}$.
 4. Compute the seven products:
- $$M1 = (A_{11} + A_{22}) \times (B_{11} + B_{22})$$
- $$M2 = (A_{21} + A_{22}) \times B_{11}$$
- $$M3 = A_{11} \times (B_{12} - B_{22})$$
- $$M4 = A_{22} \times (B_{21} - B_{11})$$
- $$M5 = (A_{11} + A_{12}) \times B_{22}$$
- $$M6 = (A_{21} - A_{11}) \times (B_{11} + B_{12})$$
- $$M7 = (A_{12} - A_{22}) \times (B_{21} + B_{22})$$
5. Compute the result sub-matrices:
- $$C_{11} = M1 + M4 - M5 + M7$$
- $$C_{12} = M3 + M5$$
- $$C_{21} = M2 + M4$$
- $$C_{22} = M1 - M2 + M3 + M6$$
6. Combine C_{11}, C_{12}, C_{21} , and C_{22} to form the final matrix C.
 7. Return matrix C.

11. Kruskal's Algorithm (Minimum Cost Spanning Tree)

ALGORITHM: KRUSKAL'S ALGORITHM (Minimum Spanning Tree)

FIND(parent[], x)

1. If $\text{parent}[x] = x$ then
- Return x.
2. Return FIND(parent, parent[x]).

KRUSKAL(edges[], V, E)

1. For $i = 0$ to $V - 1$

 parent[i] = i.

2. Sort all edges in ascending order of their weights.

3. Set cost = 0.

4. Set edgesUsed = 0.

5. For each edge (u, v, w) in the sorted list

 a. a = FIND(parent, u)

 b. b = FIND(parent, v)

 c. If $a \neq b$ then

 Print edge (u, v) with weight w.

 cost = cost + w.

 parent[a] = b. // Union operation

 edgesUsed = edgesUsed + 1.

 d. If edgesUsed = $V - 1$ then

 Break.

6. Print the minimum cost.

12. Ford-Fulkerson Algorithm (Maximum Flow)

ALGORITHM: FORD-FULKERSON ALGORITHM (Maximum Flow)

BFS(rGraph[][], s, t, parent[])

1. Initialize all visited[] = FALSE.

2. Create an empty queue.

3. Enqueue source vertex s.

4. Set visited[s] = TRUE.

5. Set parent[s] = -1.

6. While the queue is not empty do

 a. Dequeue a vertex u.

 b. For each vertex v

 If visited[v] = FALSE AND rGraph[u][v] > 0 then

 parent[v] = u.

 visited[v] = TRUE.

 Enqueue(v).

7. Return visited[t].

FORD_FULKERSON(graph[][], s, t)

1. Create the residual graph rGraph as a copy of graph.

2. Set $\text{max_flow} = 0$.
3. While $\text{BFS}(\text{rGraph}, s, t, \text{parent})$ returns TRUE do
 - a. Set $\text{path_flow} = \text{INFINITY}$.
 - b. Traverse from sink t to source s using $\text{parent}[]$.
 $\text{path_flow} = \text{MIN}(\text{path_flow}, \text{rGraph}[\text{parent}[v]][v])$
 - c. Traverse again from sink t to source s .
 $\text{rGraph}[\text{parent}[v]][v] =$
 $\text{rGraph}[\text{parent}[v]][v] - \text{path_flow}$
 $\text{rGraph}[v][\text{parent}[v]] =$
 $\text{rGraph}[v][\text{parent}[v]] + \text{path_flow}$
 - d. $\text{max_flow} = \text{max_flow} + \text{path_flow}$.
4. Return max_flow .

13. Sum of Subsets Problem (Backtracking)

ALGORITHM: SUBSET SUM (Backtracking)

PRINT_SUBSET($x[]$, $A[]$, n)

1. For $j = 0$ to $n - 1$

If $x[j] = 1$ then

Print $A[j]$.

2. Return.

SUBSET_SUM(i , currentSum)

1. If $\text{currentSum} = \text{target}$ then

Print the current subset.

Return.

2. If $i = n$ OR $\text{currentSum} > \text{target}$ then

Return.

3. Include $A[i]$ in the subset.

$x[i] = 1$.

SUBSET_SUM($i + 1$, $\text{currentSum} + A[i]$).

4. Exclude $A[i]$ from the subset.

$x[i] = 0$.

SUBSET_SUM($i + 1$, currentSum).

5. Return.

14. Hamiltonian Cycle (Backtracking)

ALGORITHM: HAMILTONIAN CYCLE (Backtracking)

IS_SAFE(v , pos)

1. If $\text{graph}[\text{path}[\text{pos} - 1]][v] = 0$ then

Return FALSE.

2. For $i = 0$ to $pos - 1$

If $path[i] = v$ then

Return FALSE.

3. Return TRUE.

HAM_CYCLE_UTIL(pos)

1. If $pos = V$ then

If $graph[path[V - 1]][path[0]] = 1$ then

Return TRUE.

Else

Return FALSE.

2. For $v = 1$ to $V - 1$

If IS_SAFE(v, pos) = TRUE then

$path[pos] = v$.

If HAM_CYCLE_UTIL($pos + 1$) = TRUE then

Return TRUE.

$path[pos] = -1$. // Backtrack

3. Return FALSE.

HAM_CYCLE()

1. Initialize all elements of $path[]$ to -1.

2. Set $path[0] = 0$.

3. If HAM_CYCLE_UTIL(1) = FALSE then

Print "No Hamiltonian Cycle Exists".

4. Else

Print the Hamiltonian Cycle by displaying
 $path[]$ followed by $path[0]$.

15. Job Sequencing with Deadlines (Greedy Method)

16. Topological Sorting (using DFS / Stack)

17. Graph Traversal using BFS and DFS

18. Topological Sorting using BFS (Kahn's Algorithm)

19. Travelling Salesman Problem (Dynamic Programming)